

WEEK 1

Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. Create a New Java Project:

- Create a new Java project named **SingletonPatternExample**.

2. Define a Singleton Class:

- Create a class named `Logger` that has a private static instance of itself.
- Ensure the constructor of `Logger` is private.
- Provide a public static method to get the instance of the `Logger` class.

3. Implement the Singleton Pattern:

- Write code to ensure that the `Logger` class follows the Singleton design pattern.

4. Test the Singleton Implementation:

- Create a test class to verify that only one instance of `Logger` is created and used across the application.

Logger.java — Singleton Class

The Logger class implements the Singleton pattern using Double-Checked Locking for thread safety.

CODE

```
// Logger.java

public class Logger {

    private static volatile Logger instance = null;

    private StringBuilder logHistory;
    private int logCount;

    private Logger() {
        logHistory = new StringBuilder();
        logCount = 0;
        System.out.println("Logger instance CREATED (this
should appear only once!)");
    }

    public static Logger getInstance() {
        if (instance == null) {
            synchronized (Logger.class) {
```

```
        if (instance == null) {  
            instance = new Logger();  
        }  
    }  
}  
return instance;  
}
```

```
public void log(String message) {  
    logCount++;  
    String logEntry = "[LOG #" + logCount + "]" +  
message;  
    logHistory.append(logEntry).append("\n");  
    System.out.println(logEntry);  
}  
public void logInfo(String message) {  
    log("[INFO] " + message);  
}
```

```
public void logWarning(String message) {  
    log("[WARN] " + message);  
}
```

```

    public void logError(String message) {
        log("[ERROR] " + message);
    }

    public void printAllLogs() {
        System.out.println("\n===== FULL LOG HISTORY
        =====");
        System.out.println(logHistory.toString());
        System.out.println("Total logs recorded: " + logCount);

        System.out.println("=====
        =\n");
    }

    public int getLogCount() {
        return logCount;
    }
}

```

SingletonTest.java — Test Class

CODE

```

// SingletonTest.java

public class SingletonTest {

```

```

public static void main(String[] args) {

    System.out.println("=====
=====");

    System.out.println(" Singleton Pattern - Logger
Demo");

    System.out.println("=====
=====\\n");

    System.out.println("--- TEST 1: Getting Logger
instances ---");

    Logger logger1 = Logger.getInstance();

    Logger logger2 = Logger.getInstance();

    Logger logger3 = Logger.getInstance();

    System.out.println("\\nAre logger1 and logger2 the
SAME object? "
        + (logger1 == logger2));

```

```
System.out.println("Are logger2 and logger3 the SAME  
object? ")
```

```
+ (logger2 == logger3));
```

```
System.out.println("logger1 memory address: " +  
logger1.hashCode());
```

```
System.out.println("logger2 memory address: " +  
logger2.hashCode());
```

```
System.out.println("logger3 memory address: " +  
logger3.hashCode());
```

```
System.out.println("\n--- TEST 2: Logging from  
different modules ---");
```

```
Logger.getInstance().logInfo("User 'john_doe' attempted  
login");
```

```
Logger.getInstance().logInfo("Login successful for  
'john_doe'");
```

```
Logger.getInstance().logInfo("Database connection  
established");
```

```
Logger.getInstance().logWarning("Query took longer  
than expected: 2.3s");
```

```
Logger.getInstance().logInfo("Payment of $99.99  
initiated");
```

```
        Logger.getInstance().logError("Payment gateway  
timeout! Retrying...");
```

```
        Logger.getInstance().logInfo("Payment successful on  
retry");
```

```
        System.out.println("\n--- TEST 3: All logs collected in  
ONE place ---");
```

```
        logger1.printAllLogs();
```

```
System.out.println("--- TEST 4: Logging from helper classes  
---");
```

```
        new UserService().createUser("Alice");
```

```
        new OrderService().placeOrder("Book", 2);
```

```
        System.out.println("\nTotal log entries across entire  
application: "
```

```
                + Logger.getInstance().getLogCount());
```

```
System.out.println("\n=====
```

```
        System.out.println(" Singleton Verified! Only 1  
instance.");
```

```
System.out.println("=====
```

```
    }
```

```
}
```

```
class UserService {
```

```
    private Logger logger = Logger.getInstance();
```

```
    public void createUser(String name) {
```

```
        logger.logInfo("UserService: Creating user " + name +  
        "");
```

```
        logger.logInfo("UserService: User " + name + " created  
        successfully");
```

```
    }
```

```
}
```

```
class OrderService {
```

```
    private Logger logger = Logger.getInstance();
```

```
    public void placeOrder(String item, int quantity) {
```

```
        logger.logInfo("OrderService: Placing order for " +  
        quantity + "x " + item);
```

```
        logger.logWarning("OrderService: Low stock alert for "  
        + item + "");
```

```
        logger.logInfo("OrderService: Order confirmed!");
```

```
    }
```

```
}
```


OUTPUT

```
F:\cognizant\week 1>javac Logger.java SingletonTest.java

F:\cognizant\week 1>java SingletonTest
=====
    Singleton Pattern - Logger Demo
=====

--- TEST 1: Getting Logger instances ---
Logger instance CREATED (this should appear only once!)

Are logger1 and logger2 the SAME object? true
Are logger2 and logger3 the SAME object? true
logger1 memory address: 112810359
logger2 memory address: 112810359
logger3 memory address: 112810359

--- TEST 2: Logging from different modules ---
[LOG #1] [INFO] User 'john_doe' attempted login
[LOG #2] [INFO] Login successful for 'john_doe'
[LOG #3] [INFO] Database connection established
[LOG #4] [WARN] Query took longer than expected: 2.3s
[LOG #5] [INFO] Payment of $99.99 initiated
[LOG #6] [ERROR] Payment gateway timeout! Retrying...
[LOG #7] [INFO] Payment successful on retry

--- TEST 3: All logs collected in ONE place ---

===== FULL LOG HISTORY =====
[LOG #1] [INFO] User 'john_doe' attempted login
[LOG #2] [INFO] Login successful for 'john_doe'
[LOG #3] [INFO] Database connection established
[LOG #4] [WARN] Query took longer than expected: 2.3s
[LOG #5] [INFO] Payment of $99.99 initiated
[LOG #6] [ERROR] Payment gateway timeout! Retrying...
[LOG #7] [INFO] Payment successful on retry

Total logs recorded: 7
=====

--- TEST 4: Logging from helper classes ---
[LOG #8] [INFO] UserService: Creating user 'Alice'
[LOG #9] [INFO] UserService: User 'Alice' created successfully
[LOG #10] [INFO] OrderService: Placing order for 2x Book
[LOG #11] [WARN] OrderService: Low stock alert for 'Book'
[LOG #12] [INFO] OrderService: Order confirmed!

Total log entries across entire application: 12

=====
    Singleton Verified! Only 1 instance.
=====
```

EXERCISE 2

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named **FactoryMethodPatternExample**.

2. Define Document Classes:

- Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.

3. Create Concrete Document Classes:

- Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.

4. Implement the Factory Method:

- Create an abstract class **DocumentFactory** with a method **createDocument()**.
- Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.

5. Test the Factory Method Implementation:

- Create a test class to demonstrate the creation of different document types using the factory method.

Document.java

CODE

```
public abstract class Document {  
    protected String fileName;  
  
    public Document(String fileName) {  
        this.fileName = fileName;  
    }  
  
    public abstract void open();  
    public abstract void save();  
    public abstract void close();  
    public abstract String getDocumentType();  
  
    public void printInfo() {  
        System.out.println("Type : " + getDocumentType());  
        System.out.println("File : " + fileName);  
    }  
}
```

WordDocument.java

```
public class WordDocument extends Document {
```

```

    public WordDocument(String fileName) {
super(fileName); }

    @Override public void open() {
System.out.println("[Word] Opening : " + fileName); }

    @Override public void save() {
System.out.println("[Word] Saving as .docx : " + fileName);
}

    @Override public void close() {
System.out.println("[Word] Closing : " + fileName); }

    @Override public String getDocumentType() { return
"Microsoft Word (.docx)"; }

}

```

PdfDocument.java

```

public class PdfDocument extends Document {

    public PdfDocument(String fileName) { super(fileName);
}

    @Override public void open() {
System.out.println("[PDF] Opening : " + fileName); }

    @Override public void save() {
System.out.println("[PDF] Saving as .pdf : " + fileName); }

    @Override public void close() { System.out.println("[PDF]
Closing : " + fileName); }

```

```
        @Override public String getDocumentType() { return  
        "PDF Document (.pdf)"; }  
    }
```

ExcelDocument.java

```
public class ExcelDocument extends Document {  
    public ExcelDocument(String fileName) {  
        super(fileName); }  
  
    @Override public void open() {  
        System.out.println("[Excel] Opening : " + fileName); }  
  
    @Override public void save() {  
        System.out.println("[Excel] Saving as .xlsx : " + fileName); }  
  
    @Override public void close() {  
        System.out.println("[Excel] Closing : " + fileName); }  
  
    @Override public String getDocumentType() { return  
        "Microsoft Excel (.xlsx)"; }  
}
```

DocumentFactory.java

```
public abstract class DocumentFactory {  
    public abstract Document createDocument(String  
        fileName);
```

```

        public void openDocument(String fileName) {

System.out.println("\n=====
==");

        Document doc = createDocument(fileName);
        doc.printInfo();
        doc.open();
        doc.save();
        doc.close();

System.out.println("=====
");
    }
}

```

WordDocumentFactory.java

```

public class WordDocumentFactory extends
DocumentFactory {

    @Override

    public Document createDocument(String fileName) {

        System.out.println("WordFactory: Creating Word
document...");

        return new WordDocument(fileName);
    }
}

```

```
    }  
}
```

PdfDocumentFactory.java

```
public class PdfDocumentFactory extends DocumentFactory  
{  
    @Override  
    public Document createDocument(String fileName) {  
        System.out.println("PdfFactory: Creating PDF  
document...");  
        return new PdfDocument(fileName);  
    }  
}
```

ExcelDocumentFactory.java

```
public class ExcelDocumentFactory extends  
DocumentFactory {  
    @Override  
    public Document createDocument(String fileName) {  
        System.out.println("ExcelFactory: Creating Excel  
document...");  
        return new ExcelDocument(fileName);  
    }  
}
```

FactoryMethodTest.java

```
public class FactoryMethodTest {  
    public static void main(String[] args) {  
  
        System.out.println("=== Document Management  
System ===");  
  
        DocumentFactory wordFactory = new  
WordDocumentFactory();  
  
        DocumentFactory pdfFactory = new  
PdfDocumentFactory();  
  
        DocumentFactory excelFactory = new  
ExcelDocumentFactory();  
  
        wordFactory.openDocument("ProjectReport");  
        pdfFactory.openDocument("Invoice_2024");  
        excelFactory.openDocument("SalesData_Q4");  
  
        System.out.println("\nAll documents created  
successfully!");  
    }  
}
```


OUTPUT

```
F:\cognizant\week 1>java FactoryMethodTest
=== Document Management System ===

=====
WordFactory: Creating Word document...
Type : Microsoft Word (.docx)
File : ProjectReport
[Word] Opening : ProjectReport
[Word] Saving as .docx : ProjectReport
[Word] Closing : ProjectReport
=====

=====
PdfFactory: Creating PDF document...
Type : PDF Document (.pdf)
File : Invoice_2024
[PDF] Opening : Invoice_2024
[PDF] Saving as .pdf : Invoice_2024
[PDF] Closing : Invoice_2024
=====

=====
ExcelFactory: Creating Excel document...
Type : Microsoft Excel (.xlsx)
File : SalesData_Q4
[Excel] Opening : SalesData_Q4
[Excel] Saving as .xlsx : SalesData_Q4
[Excel] Closing : SalesData_Q4
=====

All documents created successfully!
```

Algorithms_Data Structures

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:

- Explain Big O notation and how it helps in analyzing algorithms.
- Describe the best, average, and worst-case scenarios for search operations.

2. Setup:

- Create a class Product with attributes for searching, such as productId, productName, and category.

3. Implementation:

- Implement linear search and binary search algorithms.
- Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

- Compare the time complexity of linear and binary search algorithms.
- Discuss which algorithm is more suitable for your platform and why.

Asymptotic Notation — Theory

What is Asymptotic Notation?

Asymptotic Notation is a mathematical way to describe how the performance of an algorithm changes as the input size grows. It helps us analyze and compare algorithms without worrying about hardware speed or programming language. It focuses only on how the number of steps increases when the data size increases.

What is Big O Notation?

Big O Notation describes the worst case performance of an algorithm. It tells us the maximum number of steps an algorithm will take for a given input size n .

Big O	Name	Meaning
$O(1)$	Constant	Always takes same time, no matter the size
$O(\log n)$	Logarithmic	Steps grow very slowly as size increases
$O(n)$	Linear	Steps grow directly with size
$O(n \log n)$	Log Linear	Slightly worse than linear
$O(n^2)$	Quadratic	Steps grow very fast, bad for large data

Best, Average and Worst Case

Every algorithm has three cases depending on the input data:

Best Case — The minimum number of steps the algorithm takes. This happens when the data is in the most favorable condition.

Average Case — The expected number of steps the algorithm takes for a typical or random input. This is the most realistic measure.

Worst Case — The maximum number of steps the algorithm takes. This happens when the data is in the least favorable condition. Big O Notation always refers to worst case.

Linear Search — Case Analysis

Linear Search checks each element one by one from start to end.

Best Case — $O(1)$

The element we are searching for is at the very first position. Only 1 step needed.

Example: Searching for Product ID 103 and it is the first product in the list.

Average Case — $O(n)$

The element is somewhere in the middle of the list. On average, half the elements are checked.

Example: Searching for Product ID 105 in a list of 10 products — about 5 steps needed.

Worst Case — $O(n)$

The element is at the last position or not present at all. All n elements are checked.

Example: Searching for Product ID 110 in a list of 10 products — all 10 steps needed.

Binary Search — Case Analysis

Binary Search works only on a sorted array. It repeatedly divides the search space in half.

Best Case — $O(1)$

The element we are searching for is exactly at the middle position. Found in 1 step.

Example: Searching for Product ID 105 in a sorted list of 10 — it is the middle element, found immediately.

Average Case — $O(\log n)$

The element is found after a few divisions. Each step eliminates half the remaining elements.

Example: In 10 products, at most 4 steps. In 1000 products, at most 10 steps.

Worst Case — $O(\log n)$

The element is at the very start, very end, or not present. Maximum $\log n$ steps needed.

Example: In 1,000,000 products — Binary Search needs only 20 steps maximum

CODE

Product.java

```
public class Product {
    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String
category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    public void printProduct() {
        System.out.println("ID: " + productId + " | Name: " +
productName + " | Category: " + category);
    }
}
```

LinearSearch.java

```
public class LinearSearch {

    // Linear Search by productId
```

```
// Time Complexity: O(n) - checks every element one by one
public static Product linearSearch(Product[] products, int
targetId) {
    for (int i = 0; i < products.length; i++) {
        if (products[i].productId == targetId) {
            System.out.println("Linear Search: Found at index " + i + "
after checking " + (i + 1) + " element(s)");
            return products[i];
        }
    }
    System.out.println("Linear Search: Product not found");
    return null;
}
}
```

BinarySearch.java

```
public class BinarySearch {

    // Binary Search by productId
    // Array MUST be sorted by productId before calling this
    // Time Complexity: O(log n) - eliminates half the elements
    // each step
    public static Product binarySearch(Product[] products, int
targetId) {
        int left = 0;
        int right = products.length - 1;
        int steps = 0;

        while (left <= right) {
            steps++;
            int mid = left + (right - left) / 2;
```

```

if (products[mid].productId == targetId) {
    System.out.println("Binary Search: Found at index " + mid +
        " after " + steps + " step(s)");
    return products[mid];
} else if (products[mid].productId < targetId) {
    left = mid + 1;
} else {
    right = mid - 1;
}
}

System.out.println("Binary Search: Product not found after "
    + steps + " step(s)");
return null;
}
}

```

EcommerceSearch.java

```

public class EcommerceSearch {

    public static void main(String[] args) {

        System.out.println("--- PART 2: Product List (Unsorted - for
            Linear Search) ---");

        Product[] products = {
            new Product(103, "Laptop",    "Electronics"),
            new Product(101, "Mobile Phone", "Electronics"),
            new Product(107, "Shoes",     "Footwear"),
            new Product(102, "Headphones", "Electronics"),
            new Product(105, "T-Shirt",   "Clothing"),
            new Product(109, "Watch",     "Accessories"),
            new Product(104, "Backpack",   "Bags"),
            new Product(106, "Novel Book", "Books"),
            new Product(108, "Sunglasses", "Accessories"),
            new Product(110, "Tablet",     "Electronics")
        }
    }
}

```

```
};
```

```
System.out.println("Total Products: " + products.length);  
for (Product p : products) p.printProduct();  
System.out.println();
```

```
System.out.println("--- PART 3: Linear Search ---");  
System.out.println("Searching for Product ID = 107");  
Product result1 = LinearSearch.linearSearch(products, 107);  
if (result1 != null) result1.printProduct();  
System.out.println();
```

```
System.out.println("Searching for Product ID = 101 (first  
element - BEST case)");  
Product result2 = LinearSearch.linearSearch(products, 101);  
if (result2 != null) result2.printProduct();  
System.out.println();
```

```
System.out.println("Searching for Product ID = 110 (last  
element - WORST case)");  
Product result3 = LinearSearch.linearSearch(products, 110);  
if (result3 != null) result3.printProduct();  
System.out.println();
```

```
System.out.println("Searching for Product ID = 999 (not in  
list)");  
LinearSearch.linearSearch(products, 999);  
System.out.println();
```

```
System.out.println("--- PART 4: Sorted Product List (for  
Binary Search) ---");
```

```
Product[] sortedProducts = {  
    new Product(101, "Mobile Phone", "Electronics"),  
    new Product(102, "Headphones", "Electronics"),
```



```
new Product(103, "Laptop",    "Electronics"),
new Product(104, "Backpack",  "Bags"),
new Product(105, "T-Shirt",   "Clothing"),

new Product(106, "Novel Book", "Books"),
new Product(107, "Shoes",     "Footwear"),
new Product(108, "Sunglasses", "Accessories"),
new Product(109, "Watch",     "Accessories"),
new Product(110, "Tablet",    "Electronics")
};
```

```
System.out.println("Sorted Products:");
for (Product p : sortedProducts) p.printProduct();
System.out.println();
```

```
// -----
// PART 5 : Binary Search
// -----
```

```
System.out.println("--- PART 5: Binary Search ---");
System.out.println("Searching for Product ID = 107");
Product result4 = BinarySearch.binarySearch(sortedProducts,
107);
if (result4 != null) result4.printProduct();
System.out.println();
```

```
System.out.println("Searching for Product ID = 101 (first
element)");
Product result5 = BinarySearch.binarySearch(sortedProducts,
101);
if (result5 != null) result5.printProduct();
System.out.println();
```

```
System.out.println("Searching for Product ID = 110 (last
element)");
Product result6 = BinarySearch.binarySearch(sortedProducts,
110);
if (result6 != null) result6.printProduct();
```

```
System.out.println();
```

```
System.out.println("Searching for Product ID = 999 (not in  
list)");
```

```
BinarySearch.binarySearch(sortedProducts, 999);
```

```
System.out.println();
```

```
// -----
```

```
// PART 6 : Time Complexity Comparison
```

```
// -----
```

```
System.out.println("--- PART 6: Time Complexity Analysis -  
--");
```

```
System.out.println("+-----+-----+-----  
+-----+");
```

```
System.out.println("| Algorithm      | Best Case | Avg Case |  
Worst Case |");
```

```
System.out.println("+-----+-----+-----  
+-----+");
```

```
System.out.println("|                               Linear  
Search | O(1)  | O(n)  | O(n)  |");
```

```
System.out.println("| Binary Search | O(1)  | O(log  
n) | O(log n) |");
```

```
System.out.println("+-----+-----+-----  
+-----+");
```

```
System.out.println();
```

```
System.out.println("--- PART 7: Which is Better for E-  
Commerce? ---");
```

```
System.out.println("For 10 products      : Linear = max 10  
steps | Binary = max 4 steps");
```

```
System.out.println("For 1,000 products   : Linear = max 1000  
steps | Binary = max 10 steps");
```

```
System.out.println("For 1,000,000 products: Linear = max 1M  
steps | Binary = max 20 steps");
```

```
System.out.println();
```

```
System.out.println("CONCLUSION: Binary Search is FAR  
better for e-commerce.");
```

```

System.out.println("Reason 1 : E-commerce platforms have
millions of products.");
System.out.println("Reason 2 : Binary Search needs only
O(log n) steps.");
System.out.println("Reason 3 : Products can always be stored
sorted by ID.");
System.out.println("Reason 4 : Linear Search becomes too
slow at large scale.");
System.out.println();
System.out.println("=====
=====");
System.out.println("          Search Complete          ");
System.out.println("=====
=====");
}
}

```

OUTPUT

```

F:\cognizant\week 1>java FactoryMethodTest
=== Document Management System ===

=====
WordFactory: Creating Word document...
Type : Microsoft Word (.docx)
File : ProjectReport
[Word] Opening : ProjectReport
[Word] Saving as .docx : ProjectReport
[Word] Closing : ProjectReport
=====

=====
PdfFactory: Creating PDF document...
Type : PDF Document (.pdf)
File : Invoice_2024
[PDF] Opening : Invoice_2024
[PDF] Saving as .pdf : Invoice_2024
[PDF] Closing : Invoice_2024
=====

=====
ExcelFactory: Creating Excel document...
Type : Microsoft Excel (.xlsx)
File : SalesData_Q4
[Excel] Opening : SalesData_Q4
[Excel] Saving as .xlsx : SalesData_Q4
[Excel] Closing : SalesData_Q4
=====

All documents created successfully!

```

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. Understand Recursive Algorithms:

- **Explain the concept of recursion and how it can simplify certain problems.**

2. Setup:

- **Create a method to calculate the future value using a recursive approach.**

3. Implementation:

- **Implement a recursive algorithm to predict future values based on past growth rates.**

4. Analysis:

- **Discuss the time complexity of your recursive algorithm.**
- **Explain how to optimize the recursive solution to avoid excessive computation.**

CODE

```
public class ForecastingTool {  
  
    public static double calculateFutureValue(double  
currentAmount, double growthRate, int years) {  
    if (years == 0) {  
        return currentAmount;  
    }  
  
    double nextYearAmount = currentAmount * (1 +  
growthRate);
```

```

return calculateFutureValue(nextYearAmount, growthRate,
years - 1);
}

```

```

public static void main(String[] args) {
double initialInvestment = 1000.0;
double annualGrowthRate = 0.10;
int forecastYears = 3;

```

```

System.out.println("=====
=====");
System.out.println(" Financial Forecasting Tool Backend ");
System.out.println("=====
=====");

```

```

double finalForecast =
calculateFutureValue(initialInvestment, annualGrowthRate,
forecastYears);

```

```

System.out.printf("Initial Investment: $%.2f%n",
initialInvestment);
System.out.printf("Growth Rate: %.0f%%\n",
(annualGrowthRate * 100));
System.out.printf("Forecasted Value after %d years:
$%.2f\n", forecastYears, finalForecast);
System.out.println("=====
=====");
}
}

```

OUTPUT

```

F:\cognizant\week 1\exercise 2>javac ForecastingTool.java

F:\cognizant\week 1\exercise 2>java ForecastingTool
=====
    Financial Forecasting Tool Backend
=====
Initial Investment: $1000.00
Growth Rate: 10%
Forecasted Value after 3 years: $1331.00
=====

```

PL/SQL Exercises

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

INSERT DATA

-- 1. Create the Customers Table

```

CREATE TABLE customers (
customer_id NUMBER PRIMARY KEY,
name VARCHAR2(50),
age NUMBER,
interest_rate NUMBER,
balance NUMBER,
IsVIP VARCHAR2(10) DEFAULT 'FALSE'
);

```

-- 2. Create the Loans Table

```

CREATE TABLE loans (
loan_id NUMBER PRIMARY KEY,
customer_id NUMBER,
due_date DATE

```

);

```
-- 3. Insert Fake Customers (One young, one over 60, one rich)
INSERT INTO customers (customer_id, name, age,
interest_rate, balance) VALUES (1, 'Alice', 25, 5, 500);
INSERT INTO customers (customer_id, name, age,
interest_rate, balance) VALUES (2, 'Bob', 65, 7, 12000);
INSERT INTO customers (customer_id, name, age,
interest_rate, balance) VALUES (3, 'Charlie', 61, 6, 4000);
```

```
-- 4. Insert Fake Loans (One due tomorrow, one due next year)
INSERT INTO loans (loan_id, customer_id, due_date)
VALUES (101, 1, SYSDATE + 1);
INSERT INTO loans (loan_id, customer_id, due_date)
VALUES (102, 2, SYSDATE + 40);
```

```
COMMIT;
```

▼  CUSTOMERS
 CUSTOMER_ID
 NAME
 AGE
 INTEREST_RATE
 BALANCE
 ISVIP
▼  LOANS
 LOAN_ID
 CUSTOMER_ID
 DUE_DATE

CODE

DECLARE

```
CURSOR c_customers IS
SELECT customer_id, age, interest_rate
FROM customers;
BEGIN
FOR r_cust IN c_customers LOOP
IF r_cust.age > 60 THEN
UPDATE customers
SET interest_rate = interest_rate - 1
WHERE customer_id = r_cust.customer_id;
END IF;
END LOOP;
END;
/
```


OUTPUT

```
PL/SQL procedure successfully completed.
```

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.**

CODE

```
DECLARE
CURSOR c_vip_check IS
SELECT customer_id, balance
FROM customers;
BEGIN
FOR r_cust IN c_vip_check LOOP
IF r_cust.balance > 10000 THEN
UPDATE customers
SET IsVIP = 'TRUE'
WHERE customer_id = r_cust.customer_id;
END IF;
END LOOP;
END;
/
```

OUTPUT

```
PL/SQL procedure successfully completed.
```

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.**

CODE

```
DECLARE
CURSOR c_due_loans IS
SELECT customer_id, loan_id, due_date
FROM loans
WHERE due_date BETWEEN SYSDATE AND SYSDATE
+ 30;
BEGIN
FOR r_loan IN c_due_loans LOOP
DBMS_OUTPUT.PUT_LINE('REMINDER: Customer ID ' ||
r_loan.customer_id ||
', your payment for Loan ID ' || r_loan.loan_id ||
' is due soon!');
END LOOP;
END;
/
```

OUTPUT

```
REMINDER: Customer ID 1, your payment for Loan ID 101 is due soon!

PL/SQL procedure successfully completed.
```

Exercise 3: Stored Procedures

INSERT ROWS

```
CREATE TABLE accounts (  
  account_id NUMBER PRIMARY KEY,  
  account_type VARCHAR2(20),  
  balance NUMBER  
);
```

```
CREATE TABLE employees (  
  emp_id NUMBER PRIMARY KEY,  
  name VARCHAR2(50),  
  department VARCHAR2(30),  
  salary NUMBER  
);
```

```
INSERT INTO accounts VALUES (101, 'Savings', 5000);  
INSERT INTO accounts VALUES (102, 'Savings', 10000);  
INSERT INTO accounts VALUES (103, 'Checking', 2000);
```

```
INSERT INTO employees VALUES (1, 'John', 'IT', 60000);  
INSERT INTO employees VALUES (2, 'Sarah', 'Sales',  
50000);  
INSERT INTO employees VALUES (3, 'Mike', 'IT', 70000);
```

```
COMMIT;
```

```
1 row inserted.  
  
Elapsed: 00:00:00.003  
  
SQL> COMMIT  
  
Commit complete.
```

▼  ACCOUNTS
 ACCOUNT_ID
 ACCOUNT_TYPE
 BALANCE
▼  EMPLOYEES
 EMP_ID
 NAME
 DEPARTMENT
 SALARY

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

CODE

```
CREATE          OR          REPLACE          PROCEDURE
ProcessMonthlyInterest IS
BEGIN
UPDATE accounts
SET balance = balance * 1.01
WHERE account_type = 'Savings';

COMMIT;
END;
/
```

Procedure PROCESSMONTHLYINTEREST compiled

test Scenario 1:

```
BEGIN
ProcessMonthlyInterest;
END;
/
```

```
PL/SQL procedure successfully completed.
```

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

CODE

```
CREATE          OR          REPLACE          PROCEDURE
UpdateEmployeeBonus (
p_dept IN VARCHAR2,
p_bonus_pct IN NUMBER
) IS
BEGIN
UPDATE employees
SET salary = salary + (salary * p_bonus_pct)
WHERE department = p_dept;

COMMIT;
END;
/
```

OUTPUT

```
Procedure UPDATEEMPLOYEEBONUS compiled
```

test Scenario 2 (Give a 10% bonus to 'IT' department):

```
BEGIN
```

```
UpdateEmployeeBonus('IT', 0.10);  
END;  
/
```

PL/SQL procedure successfully completed.

Scenario 3: Customers should be able to transfer funds between their accounts.

Question: Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer

CODE

```
CREATE OR REPLACE PROCEDURE TransferFunds (  
  p_from_acc IN NUMBER,  
  p_to_acc IN NUMBER,  
  p_amount IN NUMBER  
) IS  
  v_current_balance NUMBER;  
BEGIN  
  -- Check how much money the sender has  
  SELECT balance INTO v_current_balance  
  FROM accounts  
  WHERE account_id = p_from_acc;  
  
  -- If they have enough money, make the transfer  
  IF v_current_balance >= p_amount THEN  
    -- Take money out  
    UPDATE accounts  
    SET balance = balance - p_amount  
    WHERE account_id = p_from_acc;  
  
    -- Put money in
```

```
UPDATE accounts
SET balance = balance + p_amount
WHERE account_id = p_to_acc;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Transfer Successful!');
ELSE
DBMS_OUTPUT.PUT_LINE('Error: Insufficient balance for
this transfer.');
```

END IF;

END;

/

OUTPUT

Procedure TRANSFERFUNDS compiled

test Scenario 3 (Transfer \$500 from Account 101 to 102):

```
BEGIN
TransferFunds(101, 102, 500);
END;
/
```

Transfer Successful!

PL/SQL procedure successfully completed.

